Java 21 Features

Complete Interview Preparation Guide

This document covers all major Java 21 features with:

✓ Simple Explanations
✓ Easy Code Examples
✓ Interview Questions & Answers
✓ Common Mistakes & Notes
✓ JDK 17 vs JDK 21 Differences

Quick Introduction

Java 21 was released on September 19, 2023, and is a Long-Term Support (LTS) release — the third LTS version after Java 8 and Java 11. It is the next LTS after Java 17, making it the natural upgrade target for teams currently on Java 17. Java 21 marks Oracle's commitment to releasing LTS versions every 2 years (instead of the old 3-year cycle).

Java 21 is packed with powerful features especially around concurrency (Virtual Threads, Structured Concurrency, Scoped Values — all from Project Loom), language improvements (Pattern Matching for switch finalized, Record Patterns finalized, Sequenced Collections), and performance (Generational ZGC). It includes 15 JEPs: 9 permanent features, 5 preview features, and 1 incubator.

Table of Contents

1. 1. Virtual Threads (JEP 444) — Project Loom
2. 2. Sequenced Collections (JEP 431)
3. 3. Pattern Matching for switch — Finalized (JEP 441)
4. 4. Record Patterns — Finalized (JEP 440)
5. 5. String Templates — Preview (JEP 430)
6. 6. Unnamed Patterns and Variables — Preview (JEP 443)
7. 7. Unnamed Classes and Instance Main Methods — Preview (JEP 445)
8. 8. Scoped Values — Preview (JEP 446)
9. 9. Structured Concurrency — Preview (JEP 453)
10. 10. Generational ZGC (JEP 439)
11. 11. Foreign Function & Memory API — Third Preview (JEP 442)
12. 12. Key Encapsulation Mechanism API (JEP 452)
13. 13. Deprecate Windows 32-bit & Dynamic Agent Loading (JEP 449/451)
14. 14. Migration Guide: JDK 17 to JDK 21 + Full Comparison Table

1. Virtual Threads (JEP 444) — Project Loom

Brief Explanation

Virtual Threads are a lightweight, JVM-managed alternative to the traditional OS (platform) thread. Instead of one Java thread mapping 1:1 to one OS thread, the JVM can run millions of Virtual Threads on just a handful of OS threads. They are the biggest concurrency improvement in Java since Java 5's java.util.concurrent package.

Summary

- Virtual Threads are part of Project Loom — finalized and stable in Java 21 (JEP 444)
- A traditional platform thread consumes 1–2 MB of memory; a virtual thread uses much less
- Virtual threads are managed by the JVM, not the OS
- When a virtual thread blocks (I/O, sleep), the JVM unmounts it and runs another — the carrier thread never idles
- Create with: Thread.ofVirtual().start() or Executors.newVirtualThreadPerTaskExecutor()
- Ideal for high-throughput server apps with many concurrent blocking I/O tasks
- NOT a replacement for CPU-bound work (won't speed up computation)

Detailed Description

The Problem: A traditional Java server using one-thread-per-request scales poorly. Each platform thread uses 1–2 MB of stack memory and maps to an OS kernel thread. With 10,000 concurrent requests, that is 10–20 GB of memory just for thread stacks, plus heavy OS context-switching overhead.

The Solution: Virtual Threads are extremely cheap. The JVM manages scheduling using carrier threads (a small ForkJoinPool). When a virtual thread hits a blocking call (like a database query or HTTP request), the JVM detaches it from the carrier thread and lets the carrier thread run another virtual thread. Once the blocking call completes, the virtual thread is rescheduled.

Real-world impact: A server that previously needed reactive programming (WebFlux, RxJava) to handle 100,000 concurrent requests can now use simple, synchronous, blocking code with virtual threads and achieve the same or better throughput.

Simple Examples

Creating and starting a Virtual Thread:

```
// Option 1 — Thread.ofVirtual()
Thread vt = Thread.ofVirtual().name("my-vt").start(() -> {
    System.out.println("Running in: " + Thread.currentThread());
    // blocking I/O here is fine — JVM handles it efficiently
});
vt.join();

// Option 2 — Executors (recommended for server apps)
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 100_000; i++) {
        int taskId = i;
        executor.submit(() -> {
            Thread.sleep(Duration.ofMillis(100)); // blocking — perfectly fine
            System.out.println("Task " + taskId + " done");
        });
    }
} // executor auto-closes, waits for all tasks
```

Checking if a thread is virtual:

```
Thread t = Thread.ofVirtual().start(() -> {
    System.out.println(Thread.currentThread().isVirtual()); // true
});
System.out.println(new Thread(() -> {}).isVirtual()); // false
```

Platform thread vs Virtual Thread — memory comparison:

Property	Platform Thread	Virtual Thread
Memory per thread	1 – 2 MB (stack)	A few KB (heap-allocated)
Managed by	Operating System	JVM (ForkJoinPool)
Max practical count	~Thousands	Millions
Blocking I/O behavior	OS thread idles, wastes memory	JVM unmounts it, carrier is freed
Created with	new Thread(() -> {})	Thread.ofVirtual().start(...)
CPU-bound suitability	Good	No improvement (same CPU cost)
Introduced in Java	Since Java 1	Java 21 (preview: Java 19, 20)

Interview Questions and Answers

Q1: What are Virtual Threads and why are they important in Java 21?

Answer: Virtual Threads are lightweight threads managed by the JVM instead of the OS. One carrier thread can run many virtual threads. When a virtual thread blocks, the JVM unmounts it and runs another. This allows millions of concurrent threads with low memory overhead, making it possible to write simple blocking code that scales like reactive/async code.

Q2: What is a carrier thread?

Answer: A carrier thread is a platform (OS) thread that 'carries' a virtual thread while it is actively running. The JVM maintains a small pool of carrier threads (ForkJoinPool). When a virtual thread blocks on I/O, it is unmounted from the carrier thread, freeing the carrier to run another virtual thread.

Q3: When should you NOT use Virtual Threads?

Answer: Virtual Threads do not help with CPU-bound tasks — each virtual thread still needs a CPU core to run. They are also not beneficial if your code holds synchronized locks for a long time ('pinning'), as this can block the carrier thread. Virtual Threads shine for I/O-bound tasks (HTTP calls, DB queries, file reads).

Q4: What is thread pinning with Virtual Threads?

Answer: Pinning occurs when a virtual thread cannot be unmounted from its carrier thread — typically when it is inside a synchronized block or native method. While pinned, the carrier thread is blocked, defeating the purpose of virtual threads. The fix is to replace synchronized with ReentrantLock (which supports unmounting).

Q5: How do Virtual Threads compare to reactive programming (WebFlux)?

Answer: Reactive programming (WebFlux, RxJava) achieves high concurrency by writing non-blocking async code, but has a steep learning curve and complex debugging. Virtual Threads achieve the same high concurrency using simple, synchronous blocking code — no callbacks, no reactive chains. Virtual Threads make reactive frameworks largely unnecessary for I/O throughput.

Q6: How do you enable Virtual Threads in Spring Boot?

Answer: In Spring Boot 3.2+ (with Java 21): set spring.threads.virtual.enabled=true in application.properties. This switches Tomcat and Spring's task executors to use virtual threads automatically.

Common Mistakes & Important Notes

⚠ Virtual Threads are NOT faster for CPU-bound work — they only improve I/O-bound throughput.
⚠ Avoid using synchronized blocks with Virtual Threads — use ReentrantLock instead to prevent pinning.
⚠ ThreadLocal still works with Virtual Threads but can be expensive (millions of VTs = millions of TL entries). Prefer Scoped Values.
⚠ Java 17 vs 21: Virtual Threads were PREVIEW in Java 19 and 20. They are STABLE in Java 21.
⚠ Do NOT use thread pools of Virtual Threads (newFixedThreadPool) — they are already cheap; use newVirtualThreadPerTaskExecutor().

2. Sequenced Collections (JEP 431)

Brief Explanation

Java 21 adds three new interfaces — SequencedCollection, SequencedSet, and SequencedMap — to the collections framework. These allow uniform access to the first and last elements of any ordered collection, and support reversing the collection, regardless of the specific implementation type.

Summary

- Three new interfaces: SequencedCollection<E>, SequencedSet<E>, SequencedMap<K,V>
- New methods: getFirst(), getLast(), addFirst(), addLast(), removeFirst(), removeLast()
- New method: reversed() — returns a reversed view of the collection
- All major ordered collections (List, LinkedHashSet, LinkedHashMap, Deque, SortedSet) implement these
- Solves the inconsistency problem: before Java 21, each collection type had its own first/last access method
- Finalized and stable in Java 21

The Problem It Solves

Before Java 21, getting the first or last element of different collection types required different, inconsistent approaches. List used get(0) and get(size()-1). Deque used getFirst() and getLast(). SortedSet used first() and last(). There was no uniform way to do this. Java 21 fixes this with a common interface.

To get last element of...	Before Java 21	Java 21 (SequencedCollection)
List<String>	list.get(list.size() - 1)	list.getLast()
LinkedList<String>	linkedList.getLast()	linkedList.getLast() (unchanged)
SortedSet<String>	sortedSet.last()	sortedSet.getLast()
Stream	stream().reduce((a,b)->b).get()	N/A — Stream is not a Collection

Simple Examples

```
// List (now implements SequencedCollection)
List<String> list = new ArrayList<>(List.of("A", "B", "C", "D"));

System.out.println(list.getFirst());  // A
System.out.println(list.getLast());   // D

list.addFirst("Z");  // [Z, A, B, C, D]
list.addLast("X");   // [Z, A, B, C, D, X]
list.removeFirst();   // [A, B, C, D, X]
list.removeLast();    // [A, B, C, D]

// reversed() — returns a reversed VIEW (not a copy)
List<String> rev = list.reversed();
System.out.println(rev); // [D, C, B, A]

// SequencedMap
SequencedMap<String, Integer> map = new LinkedHashMap<>();
map.put("one", 1); map.put("two", 2); map.put("three", 3);
```

```
System.out.println(map.firstEntry()); // one=1
System.out.println(map.lastEntry());  // three=3
System.out.println(map.reversed());   // {three=3, two=2, one=1}
```

Interview Questions and Answers

Q1: What are Sequenced Collections in Java 21?

Answer: Sequenced Collections introduce three new interfaces — SequencedCollection, SequencedSet, SequencedMap — that provide consistent methods (getFirst, getLast, addFirst, addLast, removeFirst, removeLast, reversed) for any ordered collection. This solves the inconsistency of different collections having different ways to access first/last elements.

Q2: What does reversed() return?

Answer: reversed() returns a reversed VIEW of the collection, not a new copy. Modifications to the reversed view are reflected in the original collection and vice versa. This is similar to how subList() works for List.

Q3: Which collections implement SequencedCollection in Java 21?

Answer: List (ArrayList, LinkedList), Deque (ArrayDeque, LinkedList), LinkedHashSet implement SequencedCollection. SortedSet and LinkedHashSet implement SequencedSet. LinkedHashMap and SortedMap implement SequencedMap.

Q4: How is this different from Java 17?

Answer: Java 17 had no SequencedCollection interfaces. Getting first/last elements required inconsistent code: list.get(0), sortedSet.first(), deque.getFirst(). Java 21 provides a uniform API across all ordered collections.

Common Mistakes & Important Notes

- ⚠ reversed() returns a VIEW — modifying it modifies the original collection.
- ⚠ getFirst() / getLast() throw NoSuchElementException on empty collections.
- ⚠ addFirst() and addLast() throw UnsupportedOperationException on immutable or unordered collections.
- ⚠ Java 17 vs 21: SequencedCollection did NOT exist in Java 17 — it is new in Java 21.

3. Pattern Matching for switch — Finalized (JEP 441)

Brief Explanation

Pattern Matching for switch was a PREVIEW feature in Java 17. Java 21 finalizes it as a stable, permanent feature. It allows switch to work with any type, use type patterns and null cases, and works with guarded patterns using the 'when' clause for conditional case labels.

Summary

- Stable in Java 21 (was preview in Java 17, 18, 19, 20)
- switch now accepts any reference type — not just int, String, enum
- 'case null' handles null values explicitly — no more NullPointerException from switch
- Guarded patterns: 'case Integer i when i > 0' — conditions on case labels
- Works seamlessly with sealed classes (exhaustive switch, no default needed)
- Works with Record Patterns for destructuring record fields in switch

Simple Examples

Type patterns with null handling:

```
static String describe(Object obj) {
    return switch (obj) {
        case null           -> "null";
        case Integer i      -> "int: " + i;
        case String s       -> "String of length " + s.length();
        case int[] arr      -> "int array of length " + arr.length;
        default             -> "Other: " + obj.getClass().getSimpleName();
    };
}
```

Guarded patterns with 'when' clause (NEW in Java 21):

```
static String classify(Object obj) {
    return switch (obj) {
        case Integer i when i < 0  -> "Negative integer";
        case Integer i when i == 0 -> "Zero";
        case Integer i             -> "Positive integer: " + i;
        case String s when s.isEmpty() -> "Empty string";
        case String s              -> "String: " + s;
        default                    -> "Other";
    };
}
```

With sealed classes — exhaustive, no default needed:

```
sealed interface Shape permits Circle, Square, Triangle {}
record Circle(double r)     implements Shape {}
record Square(double side)  implements Shape {}
record Triangle(double b, double h) implements Shape {}

static double area(Shape shape) {
    return switch (shape) {
        case Circle c          -> Math.PI * c.r() * c.r();
        case Square s          -> s.side() * s.side();
        case Triangle(var b, var h) -> 0.5 * b * h; // record pattern
        // No default! Compiler verifies all cases are covered.
    };
}
```

Interview Questions and Answers

Q1: What is new in Pattern Matching for switch in Java 21 compared to Java 17?

Answer: Java 17 had it as a PREVIEW (needed --enable-preview). Java 21 makes it STABLE. Additionally Java 21 adds: (1) guarded patterns with the 'when' clause for conditional case labels, (2) refined null handling, and (3) proper integration with Record Patterns. The 'when' clause is a key Java 21 addition.

Q2: What is a guarded pattern?

Answer: A guarded pattern adds a boolean condition to a case label using the 'when' keyword. Example: case Integer i when i > 0 matches only if the object is an Integer AND its value is greater than 0. This replaces the need for an if statement inside the case block.

Q3: What happens if switch receives null and there is no 'case null'?

Answer: Without 'case null', a switch expression/statement throws NullPointerException when it receives null — same as traditional switch behavior. Adding 'case null ->' explicitly handles null safely.

Common Mistakes & Important Notes

- ⚠ 'when' for guarded patterns is NEW in Java 21 — it did NOT exist in Java 17's preview version.
- ⚠ Java 17 vs 21: Pattern matching for switch is PREVIEW in Java 17, STABLE in Java 21.
- ⚠ Without 'case null', switch throws NullPointerException on null input — always handle null explicitly.
- ⚠ More specific patterns must come BEFORE less specific ones — the compiler enforces order.

4. Record Patterns — Finalized (JEP 440)

Brief Explanation

Record Patterns allow destructuring a Record's components directly in a pattern match expression — inside instanceof checks or switch cases. Instead of accessing record fields individually after matching, you can extract all fields directly in the pattern itself.

Summary

- Finalized in Java 21 (preview in Java 19 and 20)
- Allows destructuring of Record components inline in patterns
- Works in both instanceof expressions and switch statements/expressions
- Supports nested Record Patterns for deeply nested records
- Combines with type patterns and guarded patterns
- Makes data-processing code much more concise

Simple Examples

Without Record Patterns (Java 17 style):

```
record Point(int x, int y) {}

Object obj = new Point(3, 4);

// Java 17 — must access fields manually after matching
if (obj instanceof Point p) {
    System.out.println(p.x() + ", " + p.y());
}
```

With Record Patterns (Java 21 — destructure inline):

```
// Java 21 — destructure directly in the pattern
if (obj instanceof Point(int x, int y)) {
    System.out.println(x + ", " + y); // x and y directly in scope
}

// Using var for type inference
if (obj instanceof Point(var x, var y)) {
    System.out.println(x + ", " + y);
}
```

Record Patterns in switch:

```
sealed interface Shape permits Circle, Rectangle {}
record Circle(double radius) implements Shape {}
record Rectangle(double width, double height) implements Shape {}

static double area(Shape s) {
    return switch (s) {
        case Circle(var r)           -> Math.PI * r * r;
        case Rectangle(var w, var h) -> w * h;
    };
}
```

Nested Record Patterns:

```
record Point(int x, int y) {}
record ColoredPoint(Point point, String color) {}

Object obj = new ColoredPoint(new Point(1, 2), "red");

// Destructure nested records in one pattern
if (obj instanceof ColoredPoint(Point(var x, var y), var color)) {
```

```
    System.out.println(color + " point at " + x + "," + y);
    // Output: red point at 1,2
}
```

Interview Questions and Answers

Q1: What are Record Patterns in Java 21?

Answer: Record Patterns allow destructuring a Record's components directly in a pattern (instanceof or switch case). Instead of matching to a variable and then calling accessor methods, you extract all fields inline: 'instanceof Point(var x, var y)' gives you x and y directly without calling .x() and .y().

Q2: What is the difference between a type pattern and a record pattern?

Answer: A type pattern matches the type and binds to one variable: 'obj instanceof Point p'. A record pattern matches the type AND destructures the record's components into individual variables: 'obj instanceof Point(var x, var y)'. Record patterns go deeper into the data.

Q3: Are Record Patterns stable in Java 21?

Answer: Yes. Record Patterns were a preview feature in Java 19 and 20. Java 21 finalizes them as a stable feature.

Common Mistakes & Important Notes

- ⚠ Record Patterns are STABLE in Java 21 — they were PREVIEW in Java 19 and 20.
- ⚠ Java 17 vs 21: Record Patterns did NOT exist in Java 17 — they are purely Java 19+ features.
- ⚠ You can only destructure Records with Record Patterns — not regular classes.
- ⚠ var can be used inside Record Patterns for type inference: Point(var x, var y).

5. String Templates — Preview (JEP 430)

Brief Explanation

String Templates (preview in Java 21) allow you to embed Java expressions directly inside a string literal using the \{expression} syntax with a template processor. This is much more readable than String.format() or concatenation, especially for complex strings like SQL, JSON, or log messages.

Summary

- PREVIEW feature in Java 21 — requires --enable-preview
- Uses template processors: STR (basic interpolation), FMT (formatted), RAW
- Syntax: STR."Hello \{name}, you are \{age} years old"
- Embedded expressions \{...} can be any Java expression
- STR is the simplest processor — performs basic string interpolation
- FMT is like printf — supports format specifiers
- More readable than String.format("%s is %d years old", name, age)
- NOTE: String Templates were removed in Java 23 — this API is being redesigned

Simple Examples

```
// Requires --enable-preview to compile and run

String name = "Alice";
int age = 30;

// Old way — hard to match placeholders to arguments
String old = String.format("%s is %d years old", name, age);

// Java 21 STR template processor — named, readable
String result = STR."\{name} is \{age} years old";
// Output: Alice is 30 years old

// Expressions inside \{...}
String s2 = STR."2 + 2 = \{2 + 2}";      // 2 + 2 = 4
String s3 = STR."Upper: \{name.toUpperCase()}"; // Upper: ALICE

// FMT processor — like printf
double price = 19.99;
String formatted = FMT."Price: $%.2f\{price}"; // Price: $19.99

// Multi-line text block with STR
String json = STR."""
        {
            \"name\": \"\{name}\",
            \"age\": \{age}
        }
        """;
```

Interview Questions and Answers

Q1: What are String Templates in Java 21?

Answer: String Templates (JEP 430, Preview) allow embedding Java expressions inside string literals using the \{expression} syntax with a template processor. STR."Hello \{name}" is cleaner than "Hello " + name or String.format("Hello %s", name). The STR processor does simple interpolation; FMT supports format specifiers.

Q2: What is the STR template processor?

Answer: STR is a built-in template processor that performs basic string interpolation. It replaces each \{expression} with the result of evaluating that expression. It is automatically imported and available — you do not need to import it. Example: STR."Hello \{name}" replaces \{name} with the value of the name variable.

Q3: Are String Templates stable in Java 21?

Answer: No. String Templates are a PREVIEW feature in Java 21 (JEP 430). They require --enable-preview. Importantly, the feature was withdrawn from Java 23 for redesign — so the Java 21 API may change significantly in future versions.

Q4: How are String Templates different from Text Blocks?

Answer: Text Blocks (stable since Java 15) are for writing multi-line string literals cleanly — they do not support variable interpolation. String Templates add interpolation (\{expression}) on top of both single-line strings and text blocks. They serve complementary purposes.

Common Mistakes & Important Notes

⚠ String Templates are PREVIEW in Java 21 — requires --enable-preview flag.
⚠ IMPORTANT: String Templates were WITHDRAWN in Java 23 for redesign — the API may change significantly.
⚠ Java 17 vs 21: String Templates are NEW in Java 21. Java 17 has no equivalent.
⚠ Do not use String Templates in production Java 21 code — the API is not yet finalized.

6. Unnamed Patterns and Variables — Preview (JEP 443)

Brief Explanation

Java 21 introduces the underscore _ as an unnamed pattern variable placeholder. When you match a type in a pattern but don't need to use the bound variable, you can use _ instead of giving it a name. This reduces clutter and signals intent — 'I matched this type, but I don't need the value.'

Summary

- PREVIEW in Java 21 (JEP 443)
- Use _ (underscore) when you need to match a type but don't care about the variable
- Works in: instanceof patterns, switch case patterns, catch blocks, for-each loops
- Makes code cleaner — eliminates unused variable warnings
- Multiple _ in the same scope is allowed (unlike normal variables)

Simple Examples

```
// Without unnamed patterns — 'e' is never used
try {
    riskyOperation();
} catch (IOException e) {  // e is declared but unused
    System.out.println("IO error occurred");
}

// With unnamed variable — cleaner, no unused-variable warning
try {
    riskyOperation();
} catch (IOException _) {  // _ signals 'I don't need this'
    System.out.println("IO error occurred");
}

// Unnamed pattern in switch
Object obj = "hello";
switch (obj) {
    case Integer _ -> System.out.println("It's an integer");
    case String _  -> System.out.println("It's a string");
    default        -> System.out.println("Something else");
}

// In for-each when index matters but element doesn't
int count = 0;
for (var _ : list) { count++; }
```

Interview Questions and Answers

Q1: What are Unnamed Patterns and Variables in Java 21?

Answer: Unnamed Patterns and Variables (JEP 443, Preview) allow using _ as a placeholder when you match a type in a pattern but don't need to use the bound variable. This eliminates unnecessary variable names and signals to readers that the matched value is intentionally ignored.

Q2: Why was _ (underscore) chosen for unnamed variables?

Answer: Underscore is commonly used across many languages (Python, Scala, Haskell) to indicate a throwaway value. In Java 8, a warning was added for single-underscore variable names. Java 9 made _ a reserved keyword. Java 21 gives it a specific meaning: unnamed/ignored variable.

Common Mistakes & Important Notes

- ⚠ This is a PREVIEW feature in Java 21 — requires --enable-preview.
- ⚠ You cannot READ the value of _ — it is write-only (declaration only).
- ⚠ Multiple _ variables are allowed in the same scope (unlike named variables).
- ⚠ Java 17 vs 21: Unnamed Patterns did NOT exist in Java 17.

7. Unnamed Classes and Instance Main Methods — Preview (JEP 445)

Brief Explanation

Java 21 (Preview) allows writing Java programs without a class declaration and with a simpler main() method signature. This lowers the barrier to entry for beginners, scripting, and quick tests. You can write a Java program with just a main() method — no 'public class', no 'public static void main(String[] args)'.

Summary

- PREVIEW in Java 21 (JEP 445)
- Unnamed class: a Java file without an explicit class declaration
- Instance main method: main() can be an instance method (not static) with no parameters
- The Java launcher automatically wraps unnamed source files in an implicit class
- Designed for beginners, scripting, and quick prototypes
- Cannot be imported by or reference named classes

Simple Examples

Traditional Java hello world (verbose):

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Java 21 unnamed class (preview) — no class, no static, no args:

```
// Just save as HelloWorld.java and run with --enable-preview
void main() {
    System.out.println("Hello, World!");
}
```

Instance main method with simpler signatures — all are valid:

```
// All of these are valid main method signatures in Java 21 (preview):

static void main(String[] args) { ... } // traditional
static void main()              { ... } // no args
void main(String[] args)        { ... } // instance, with args
void main()                     { ... } // simplest form
```

Interview Questions and Answers

Q1: What are Unnamed Classes in Java 21?

Answer: Unnamed Classes (JEP 445, Preview) allow writing Java programs without explicitly declaring a class. The compiler wraps the file content in an implicit class. This makes simple programs and scripts much shorter and easier to write, especially for beginners.

Q2: Is this replacing the traditional public class and public static void main?

Answer: No. The traditional form still works and is required for production code, libraries, and reusable classes. Unnamed Classes are for simple programs, scripts, and learning. They cannot be imported by or reference other named classes.

Common Mistakes & Important Notes

- ⚠ PREVIEW in Java 21 — requires --enable-preview. This became more stable in Java 23/25.
- ⚠ Unnamed classes cannot be imported or reference named classes — for standalone scripts only.
- ⚠ This is controversial — some developers worry it teaches bad habits to newcomers.
- ⚠ Java 17 vs 21: Does not exist in Java 17. New in Java 21 (preview).

8. Scoped Values — Preview (JEP 446)

Brief Explanation

Scoped Values are a safer, more efficient alternative to ThreadLocal for sharing immutable data within a thread and its child threads. They are designed specifically to work well with Virtual Threads. A scoped value is written once and is then immutable — no one can accidentally change the value in a child scope.

Summary

- PREVIEW in Java 21 (JEP 446)
- Alternative to ThreadLocal — designed for Virtual Threads
- A scoped value is written ONCE and is immutable within its scope
- Data is automatically available to child threads (unlike ThreadLocal)
- Cleaner than ThreadLocal: no get/set/remove lifecycle to manage
- Use ScopedValue.where(VALUE, data).run(...) to bind and use
- Part of Project Loom along with Virtual Threads and Structured Concurrency

ThreadLocal vs Scoped Values

Property	ThreadLocal	ScopedValue (Java 21)
Mutability	Mutable — can be changed anytime	Immutable within scope — write once
Lifetime	Lives as long as the thread	Bounded to a specific scope
Inheritance	Optional (InheritableThreadLocal)	Automatically inherited by child threads
Virtual Thread cost	Expensive (copy per VT)	Lightweight and efficient
Safety	Can be forgotten/leaked	Automatically cleaned up after scope
API	get(), set(), remove()	ScopedValue.where().run()

Simple Examples

```
import java.lang.ScopedValue; // Preview — needs --enable-preview

// Define a scoped value (usually as a static final field)
static final ScopedValue<String> CURRENT_USER = ScopedValue.newInstance();

// Bind a value and run code within that scope
ScopedValue.where(CURRENT_USER, "Alice").run(() -> {
    processRequest(); // CURRENT_USER is "Alice" inside here
});

void processRequest() {
    String user = CURRENT_USER.get(); // Alice
    System.out.println("Processing for: " + user);
    // Even methods called from here can access CURRENT_USER
    validatePermissions();
}
```

```
// After the run() block, CURRENT_USER reverts to unbound
// CURRENT_USER.get() here would throw NoSuchElementException
```

Real-world use case — passing request context to nested calls:

```
// Web framework: set user context for each request
static final ScopedValue<String> REQUEST_ID = ScopedValue.newInstance();
static final ScopedValue<User> LOGGED_USER  = ScopedValue.newInstance();

// In request handler:
ScopedValue.where(REQUEST_ID, "req-123")
           .where(LOGGED_USER, currentUser)
           .run(this::handleRequest);

// Deep in the call chain — no need to pass parameters
void handleRequest() {
    String reqId = REQUEST_ID.get(); // automatically available
    User user = LOGGED_USER.get();   // automatically available
}
```

Interview Questions and Answers

Q1: What are Scoped Values in Java 21?

Answer: Scoped Values (JEP 446, Preview) are an alternative to ThreadLocal for sharing immutable context data within a thread and its child threads. They are defined with ScopedValue.newInstance() and bound using ScopedValue.where(value, data).run(...). Within the run() block, the value is accessible; outside it, it is not.

Q2: Why are Scoped Values better than ThreadLocal for Virtual Threads?

Answer: ThreadLocal keeps a mutable value that lives as long as the thread. With millions of virtual threads, each having its own ThreadLocal entries, memory usage becomes expensive. Scoped Values are immutable and bounded to a scope — they are automatically cleaned up, making them much more efficient with Virtual Threads.

Q3: Can you modify a Scoped Value after binding?

Answer: No. A Scoped Value is immutable after binding with where(). If you need a different value in a nested scope, you can rebind it: ScopedValue.where(VALUE, newValue).run(...) creates a new binding that shadows the outer one within the inner scope.

Common Mistakes & Important Notes

- ⚠ PREVIEW in Java 21 — requires --enable-preview.
- ⚠ ScopedValue.get() throws NoSuchElementException if called outside its binding scope.
- ⚠ Scoped Values are immutable — use isBound() to check if a value is currently bound.
- ⚠ Java 17 vs 21: Scoped Values did NOT exist in Java 17 — new in Java 20 (incubator), Java 21 (preview).

9. Structured Concurrency — Preview (JEP 453)

Brief Explanation

Structured Concurrency treats a group of related concurrent tasks as a single unit of work. When you fork multiple tasks inside a StructuredTaskScope, they all complete (or fail) together before the scope exits. This solves the chaos of managing multiple threads manually — cancellation, error handling, and lifetime are all handled automatically.

Summary

- PREVIEW in Java 21 (JEP 453, was incubator in Java 19/20)
- Main class: StructuredTaskScope (in java.util.concurrent)
- Fork subtasks with scope.fork(callable) — returns a Subtask handle
- scope.join() waits for all forked tasks to complete
- ShutdownOnFailure: if any subtask fails, all others are cancelled
- ShutdownOnSuccess: completes as soon as any subtask succeeds
- Part of Project Loom — designed to work with Virtual Threads
- Makes concurrent code easier to read, debug, and maintain

The Problem It Solves

Before Structured Concurrency, running multiple concurrent tasks meant manually managing CompletableFutures or ExecutorService. If one task failed, you had to manually cancel others. Error handling was scattered. Structured Concurrency enforces a tree-shaped task hierarchy — parent tasks outlive child tasks, ensuring clean lifecycle management.

Simple Examples

ShutdownOnFailure — all tasks must succeed:

```
import java.util.concurrent.StructuredTaskScope;

String fetchUserAndOrder(long id) throws Exception {
    try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {

        // Fork two concurrent tasks
        var userTask  = scope.fork(() -> fetchUser(id));
        var orderTask = scope.fork(() -> fetchLatestOrder(id));

        scope.join()           // wait for both tasks
             .throwIfFailed(); // throw if any task failed

        // Both completed successfully
        return userTask.get() + ": " + orderTask.get();
    }
    // scope closes: any unfinished tasks are cancelled automatically
}
```

ShutdownOnSuccess — first success wins (e.g., fastest server):

```
String fetchFastest() throws Exception {
    try (var scope = new StructuredTaskScope.ShutdownOnSuccess<String>()) {
        scope.fork(() -> fetchFromServer1());
        scope.fork(() -> fetchFromServer2());
        scope.fork(() -> fetchFromServer3());

        scope.join(); // waits until first success
```

```
        return scope.result(); // result of fastest task
    }
}
```

Feature	Traditional Threads/Futures	Structured Concurrency (Java 21)
Task lifetime	Tasks can outlive their parent	Tasks always complete before scope exits
Cancellation	Manual — complex, error-prone	Automatic when scope closes
Error handling	Scattered across callbacks	Centralized: throwIfFailed()
Debugging	Hard — threads are independent	Clear tree hierarchy in stack traces
Code style	Callback / reactive	Sequential, readable

Interview Questions and Answers

Q1: What is Structured Concurrency in Java 21?

Answer: Structured Concurrency (JEP 453, Preview) provides an API where groups of concurrent tasks are treated as a single unit of work. Using StructuredTaskScope, you fork subtasks and join them. If any task fails (ShutdownOnFailure), all others are cancelled. Tasks never outlive their parent scope — lifecycle management is automatic.

Q2: What is the difference between ShutdownOnFailure and ShutdownOnSuccess?

Answer: ShutdownOnFailure cancels all remaining tasks if any one task fails — used when ALL tasks must succeed. ShutdownOnSuccess cancels all remaining tasks as soon as any one task succeeds — used when you only need the FIRST result (e.g., fastest server response).

Q3: How does Structured Concurrency relate to Virtual Threads?

Answer: Structured Concurrency is designed to work with Virtual Threads — StructuredTaskScope.fork() creates virtual threads by default. This combination allows spawning thousands of concurrent tasks cheaply, with clean lifecycle management and structured error handling.

Q4: How is Structured Concurrency different from CompletableFuture?

Answer: CompletableFuture creates a flat, unstructured set of tasks with complex chaining for error handling. Structured Concurrency creates a tree hierarchy where parent tasks always outlive children. Cancellation and error propagation are automatic. The code reads sequentially, making it much easier to understand and debug.

Common Mistakes & Important Notes

⚠ PREVIEW in Java 21 — requires --enable-preview.
⚠ Always use try-with-resources for StructuredTaskScope — it implements AutoCloseable.
⚠ scope.join() must be called before accessing subtask results with subtask.get().
⚠ Java 17 vs 21: Structured Concurrency did NOT exist in Java 17 — it is new in Java 19 (incubator), 21 (preview).

10. Generational ZGC (JEP 439)

Brief Explanation

Java 21 upgrades the Z Garbage Collector with generational support. Generational ZGC maintains separate generations for young (short-lived) and old (long-lived) objects, allowing the GC to collect young objects more frequently and efficiently. This significantly improves performance over the original ZGC.

Summary

- Stable in Java 21 (JEP 439)
- ZGC was introduced in Java 11 (experimental), production-ready in Java 15
- Generational ZGC adds young/old generations to ZGC (like G1GC but with ZGC's low-latency model)
- Young objects are collected more frequently — most objects die young (generational hypothesis)
- Better throughput AND maintained low-latency (< 10ms pauses)
- Enable in Java 21: -XX:+UseZGC -XX:+ZGenerational
- Default ZGC (non-generational) still available in Java 21; Generational will become the default in future

Simple Examples

```
# Enable Generational ZGC in Java 21
java -XX:+UseZGC -XX:+ZGenerational MyApp

# Non-generational ZGC (old default, still works)
java -XX:+UseZGC MyApp

# With heap size
java -XX:+UseZGC -XX:+ZGenerational -Xmx8g MyApp
```

GC	Java Version	Pause Times	Generational?	Best For
Serial GC	All	Long	No	Single-threaded, tiny apps
Parallel GC	All	Medium	Yes	Throughput-focused batch apps
G1GC	Java 9 (def)	Short	Yes	Balanced latency & throughput
ZGC	Java 11+	< 1ms	Java 21+	Low latency, large heaps
Epsilon GC	Java 11+	None	No	Testing only — no GC at all

Interview Questions and Answers

Q1: What is Generational ZGC in Java 21?

Answer: Generational ZGC (JEP 439) upgrades ZGC with separate young and old generations. Since most objects die young (the generational hypothesis), collecting young objects more frequently reduces GC overhead while maintaining ZGC's low-latency guarantees. It provides better throughput than the original single-generation ZGC.

Q2: How do you enable Generational ZGC in Java 21?

Answer: Use -XX:+UseZGC -XX:+ZGenerational. In Java 21 you must explicitly opt in. In future Java versions, Generational ZGC will become the default ZGC mode.

Q3: What is the generational hypothesis in GC?

Answer: The generational hypothesis states that most objects die young (are short-lived). Generational GCs exploit this by collecting young objects frequently (minor GC) and old objects less often (major GC). This makes GC more efficient because collecting a small young generation is faster than scanning the entire heap.

Common Mistakes & Important Notes

- ⚠ In Java 21, Generational ZGC must be enabled explicitly: -XX:+ZGenerational
- ⚠ Java 17 vs 21: ZGC was stable in Java 15+, but Generational ZGC is new in Java 21.
- ⚠ In Java 23, Generational ZGC became the default — non-generational ZGC was deprecated.
- ⚠ ZGC still has slightly lower throughput than G1GC for some workloads — test before switching.

11. Foreign Function & Memory API — Third Preview (JEP 442)

Brief Explanation

The Foreign Function & Memory API allows Java to call native C/C++ code and manage off-heap memory without JNI. In Java 21 it is in its Third Preview, with an improved and more stable API compared to its Java 17 incubator stage. It becomes stable in Java 22 (JEP 454).

Summary

- Third Preview in Java 21 (JEP 442) — was Incubator in Java 17
- Replaces JNI (Java Native Interface) — much easier and safer
- MemorySegment: represents a region of off-heap or on-heap memory
- MethodHandle-based foreign function calls — no C headers needed
- Arena controls the lifetime of off-heap memory allocations
- Became STABLE in Java 22 (JEP 454) — use Java 22+ for production
- Built under Project Panama

Interview Questions and Answers

Q1: What is the Foreign Function & Memory API in Java 21?

Answer: The FF&M API (JEP 442, Third Preview in Java 21) allows Java code to directly call native C functions and manage off-heap memory without using JNI. It uses MethodHandle-based lookups and MemorySegment for memory management. It is safer than JNI (bounds-checked memory access) and requires no C header files.

Q2: What is the status of this API in Java 17 vs Java 21 vs Java 22?

Answer: Java 17: Incubator (very experimental). Java 21: Third Preview (stable enough for testing, API mostly finalized). Java 22: Stable/Permanent (production-ready). For production use, Java 22+ is recommended.

Q3: What is an Arena in the FF&M API?

Answer: An Arena controls the lifetime of off-heap memory. Memory allocated within an Arena is automatically freed when the Arena is closed (try-with-resources). This prevents memory leaks that were common with manual JNI memory management.

> ⚠ Third Preview in Java 21 — needs --enable-preview.
> ⚠ Use Java 22+ (stable) for production use of this API.
> ⚠ JNI still works in Java 21 — FF&M is the modern replacement, not yet mandatory.

12. Key Encapsulation Mechanism API & Other Notable JEPs

Key Encapsulation Mechanism API (JEP 452) — Stable

The Key Encapsulation Mechanism (KEM) API allows Java applications to implement modern key exchange algorithms — including post-quantum cryptography algorithms being standardized by NIST. It provides a standard API for wrapping and unwrapping symmetric keys using asymmetric cryptography.

Summary

- Stable in Java 21 (JEP 452)
- Provides javax.crypto.KEM class for key encapsulation operations
- Supports RSA-KEM, Elliptic Curve Integrated Encryption Scheme (ECIES)
- Designed to support post-quantum cryptography (PQC) algorithms
- Used in TLS key exchange and hybrid encryption schemes

Q1: What is the Key Encapsulation Mechanism API in Java 21?

Answer: The KEM API (JEP 452) provides a standard interface for key encapsulation — a cryptographic technique where a symmetric key is securely wrapped using an asymmetric key. It supports modern algorithms including post-quantum cryptography candidates. The main class is javax.crypto.KEM.

Deprecate the Windows 32-bit x86 Port for Removal (JEP 449)

Java 21 deprecates (for removal) the Windows 32-bit x86 JDK port. Windows 10 (the last version supporting 32-bit) reaches end of life in October 2025. After that, there is no reason to maintain a 32-bit Windows JVM. An error message is shown when configuring a 32-bit build on Windows.

Q1: Why is the Windows 32-bit port being deprecated?

Answer: Windows 10 (Version 22H2) is the last Windows version supporting 32-bit operation, and it reached end of life in October 2025. After this, 32-bit Java on Windows has no practical use case, so Oracle decided to deprecate and eventually remove this port to reduce maintenance costs.

Prepare to Disallow Dynamic Loading of Agents (JEP 451)

Java 21 adds a warning when Java agents (like profilers and monitoring tools) are loaded dynamically into a running JVM. In the future, this will become an error. Agents should be loaded at JVM startup using -javaagent or --agentlib flags instead.

Q1: What is JEP 451 about?

Answer: JEP 451 prepares for restricting dynamic agent loading (loading monitoring/profiling agents into a running JVM). Java 21 shows a warning. Future releases will make it an error. This improves security by preventing unexpected code injection into running JVMs. Tools like VisualVM and profilers must adapt to load at startup.

> ⚠ JEP 451 only shows a WARNING in Java 21. Future versions will make it an ERROR.
> ⚠ If your monitoring tool loads agents dynamically, update it or add --XX:+EnableDynamicAgentLoading as a temporary flag.
> ⚠ Java 17 vs 21: Both JEP 449 and JEP 451 are new in Java 21.

13. Migration Guide: JDK 17 to JDK 21

Brief Explanation

Migrating from Java 17 to Java 21 is the smoothest of all major migrations. Since Java 21 is the next LTS after Java 17, Oracle has ensured high backward compatibility. There are no removed Java EE modules (that already happened in Java 11). The main concerns are: finalized APIs replacing previews, new deprecations, and agent loading warnings.

Area	Java 17 Behavior	Java 21 Behavior	Action Required
Pattern Matching for switch	Preview — needs --enable-preview	STABLE — no flag needed	Remove --enable-preview if using this feature only
Record Patterns	Preview (Java 19/20)	STABLE in Java 21	Remove --enable-preview
Virtual Threads	Not available	STABLE in Java 21	Start using for I/O-bound concurrency
Sequenced Collections	Not available	STABLE in Java 21	Replace workarounds with getFirst/getLast
--illegal-access flag	Already REMOVED in Java 17	Still removed	Nothing — already handled
Dynamic Agent Loading	Allowed silently	Warning issued in Java 21	Plan for future restriction; load agents at startup
Windows 32-bit port	Available	Deprecated for removal	Migrate to 64-bit if using 32-bit Windows JDK
ZGC	Stable (single gen)	Generational ZGC available	Add -XX:+ZGenerational if using ZGC
String Templates	Not available	Preview in Java 21	Optional — requires --enable-preview
Scoped Values	Not available	Preview in Java 21	Optional — for replacing ThreadLocal in VT apps
Structured Concurrency	Not available	Preview in Java 21	Optional — replaces CompletableFuture patterns

Migration Checklist

- Step 1: Upgrade JDK to 21 and compile — most Java 17 code compiles directly
- Step 2: Remove --enable-preview flags if your only preview features are now stable (Pattern Matching for switch, Record Patterns)
- Step 3: Check for dynamic agent loading warnings (JEP 451) in logs — update profiling tools
- Step 4: Upgrade Spring Boot to 3.2+ to get Virtual Thread support automatically
- Step 5: Identify I/O-bound code that could benefit from Virtual Threads
- Step 6: Replace ThreadLocal with ScopedValues in Virtual Thread-heavy code (optional, preview)
- Step 7: Enable Generational ZGC if you use ZGC (-XX:+ZGenerational)
- Step 8: Update build tools (Maven 3.9+, Gradle 8.4+) for Java 21 compatibility

Interview Questions and Answers

Q1: How difficult is the migration from Java 17 to Java 21?

Answer: Very smooth — this is the easiest major Java migration. Java 17 already removed Java EE modules and the --illegal-access flag. Java 21 is backward-compatible with well-written Java 17 code. The main changes are: pattern matching for switch and record patterns are now stable (no more --enable-preview needed), and new deprecation warnings for dynamic agent loading.

Q2: What is the biggest new feature to adopt when moving to Java 21?

Answer: Virtual Threads (JEP 444) — they are stable, free to use, and can dramatically improve the scalability of I/O-bound server applications. Any Spring Boot 3.2+ application can enable them with one configuration line: spring.threads.virtual.enabled=true.

Q3: What happens to preview features from Java 17 in Java 21?

Answer: Pattern Matching for switch (preview in Java 17) is STABLE in Java 21. Record Patterns (preview in Java 19/20) are STABLE in Java 21. Any --enable-preview flags used specifically for these features can be removed.

14. Full Java Version Comparison: Java 8 → 11 → 17 → 21

Complete Feature Matrix

Feature	Java 8	Java 11	Java 17	Java 21
Lambda & Streams	✓ Introduced	✓	✓	✓
Optional class	✓ Introduced	✓ + isEmpty()	✓	✓
HTTP Client API	✗	✓ Stable	✓	✓
Text Blocks	✗	✗	✓ Stable (Java 15)	✓
Records	✗	✗	✓ Stable (Java 16)	✓
Sealed Classes	✗	✗	✓ Stable	✓
Pattern Matching instanceof	✗	✗	✓ Stable	✓
Pattern Matching switch	✗	✗	Preview only	✓ STABLE
Record Patterns	✗	✗	✗	✓ STABLE
Guarded Patterns (when clause)	✗	✗	✗	✓ STABLE
Switch Expressions (->)	✗	✗	✓ (Java 14)	✓
Virtual Threads	✗	✗	✗	✓ STABLE
Sequenced Collections	✗	✗	✗	✓ STABLE
Structured Concurrency	✗	✗	✗	Preview
Scoped Values	✗	✗	✗	Preview
String Templates	✗	✗	✗	Preview
Unnamed Patterns (_)	✗	✗	✗	Preview
Generational ZGC	✗	✗	✗	✓ STABLE
ZGC (non-generational)	✗	Experimental	✓ Stable	✓
var keyword	✗	✓ (local + lambda)	✓	✓
JAXB / Java EE modules	✓ Built-in	✗ Removed	✗ Removed	✗ Removed
Nashorn JS Engine	✓	Deprecated	✗ Removed	✗ Removed
Apple Silicon (AArch64)	✗	✗	✓ Native	✓
Spring Boot 3 compatible	✗	✗	✓ Required (min)	✓ Recommended
LTS Release	✓	✓	✓	✓

Project Loom Feature Summary (New in Java 21)

Project Loom is an OpenJDK project that introduced three complementary features to revolutionize Java concurrency. All three are available in Java 21:

Project Loom Feature	Java 21 Status	Purpose	Replaces
Virtual Threads (JEP 444)	STABLE	Lightweight threads for I/O concurrency	Thread pools, reactive frameworks
Structured Concurrency (JEP 453)	Preview	Treat related tasks as one unit of work	CompletableFuture, ExecutorService
Scoped Values (JEP 446)	Preview	Immutable thread-local context sharing	ThreadLocal

JDK 21 JEP Summary — All 15 JEPs

JEP	Feature	Status	Category
430	String Templates	Preview	Language
431	Sequenced Collections	Stable/Permanent	API
439	Generational ZGC	Stable/Permanent	Performance
440	Record Patterns	Stable/Permanent	Language
441	Pattern Matching for switch	Stable/Permanent	Language
442	Foreign Function & Memory API	Third Preview	API
443	Unnamed Patterns and Variables	Preview	Language
444	Virtual Threads	Stable/Permanent	Concurrency
445	Unnamed Classes & Instance Main Methods	Preview	Language
446	Scoped Values	Preview	Concurrency
448	Vector API	Sixth Incubator	Performance
449	Deprecate Windows 32-bit x86 Port	Stable/Permanent	Platform
451	Prepare to Disallow Dynamic Agent Loading	Stable/Permanent	Security
452	Key Encapsulation Mechanism API	Stable/Permanent	Security
453	Structured Concurrency	Preview	Concurrency

Common Mistakes & Important Notes

⚠ Virtual Threads are STABLE in Java 21 — the #1 feature to use for I/O-bound servers.
⚠ Structured Concurrency and Scoped Values are PREVIEW in Java 21 — needs --enable-preview.
⚠ String Templates (Preview in Java 21) were WITHDRAWN in Java 23 — do not rely on this API.
⚠ Pattern Matching for switch + Record Patterns are NOW STABLE — remove --enable-preview flags.
⚠ Spring Boot 3.2+ with Java 21 gives you Virtual Threads for free via one config property.
⚠ Sequenced Collections (getFirst/getLast/reversed) work immediately — no flags needed.
⚠ Generational ZGC needs -XX:+ZGenerational in Java 21; becomes default in Java 23+.